

Curso: Compiladores

Docente: Mgtr. Moises Alonso

Facultad de Ingeniería

Ingeniería Informática y Sistemas

Universidad Rafael Landívar

Avenger.g4

Documentación Técnica de Gramática

Fase II - Análisis Sintáctico, Semántico y Traducción

Nery Samuel Hernández Herrera - 1098824

Gabriel Emilio Toyom Jiménez - 1051524

Marc Wilhelm Schaub García - 1243424

1. Introducción

En este documento describimos el diseño e implementación de la segunda fase del compilador para el lenguaje de programación temático AvengerScript. Tras haber completado la definición de la gramática libre de contexto y el analizador léxico en la Fase I, el proyecto avanzó hacia la construcción del analizador sintáctico (Parser), el análisis semántico y la traducción dirigida por la sintaxis mediante la herramienta ANTLR4.

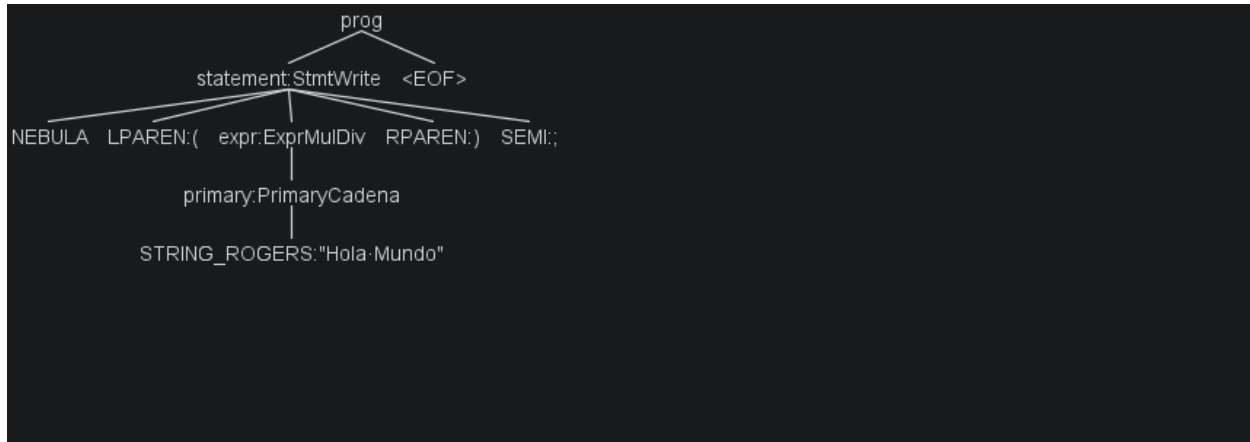
A diferencia de los compiladores tradicionales que reducen el código a lenguaje máquina o ensamblador, el objetivo en esta fase fue actuar como un transpilador. Se procesó el flujo de tokens para construir un Árbol de Análisis Sintáctico (AST) y, al recorrer sus nodos, generar dinámicamente código equivalente y ejecutable en un lenguaje de alto nivel destino, específicamente Java.

2. Arquitectura del Parser y Árbol de Expresiones (AST)

El analizador sintáctico de AvengerScript toma el flujo de tokens (lexemas) validados y verifica que cumplan con la estructura gramatical definida en el archivo Avenger.g4. Al validar una cadena, ANTLR genera automáticamente un árbol de expresiones que jerarquiza las operaciones.

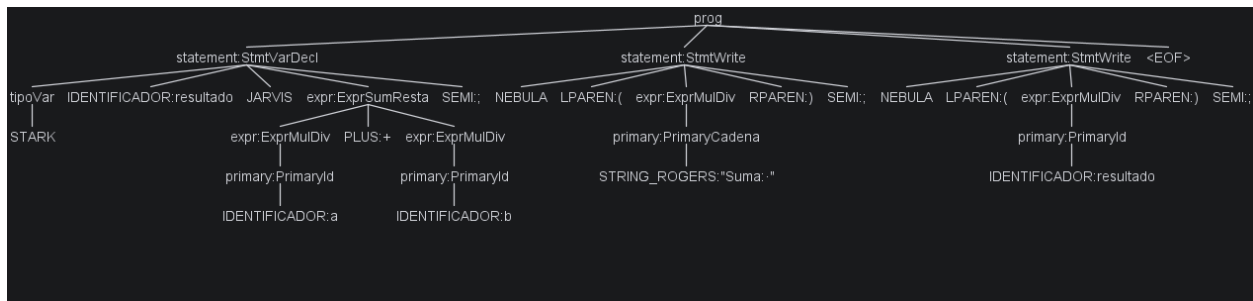
A continuación, se presentan ejemplos visuales de los árboles generados por el Parser para distintas producciones del lenguaje:

Figura 1: AST para una instrucción de salida simple (Hola Mundo)



Justificación: El árbol muestra cómo la regla principal prog deriva en un statement de tipo StmtWrite (token léxico NEBULA), el cual agrupa los paréntesis y procesa la cadena literal STRING_ROGERS como una expresión primaria.

Figura 2: AST para declaración, suma matemática y salida de datos



Justificación: En este árbol complejo se observa la precedencia aritmética. La declaración (StmtVarDecl) evalúa una expresión de suma (ExprSumResta), la cual a su vez tiene como hijos a dos identificadores independientes. Posteriormente, las sentencias de escritura (StmtWrite) acceden a la variable resultado.

3. Tabla de Símbolos y Análisis Semántico

Para interactuar con el AST generado, el equipo optó por el patrón de diseño Visitor, implementando la clase EvalVisitor que hereda de AvengerBaseVisitor<String>. Para dotar de contexto a la traducción, se

implementó una Tabla de Símbolos persistente en memoria a través de una `List<Variable>`.

Cada entrada en la tabla registra cuatro propiedades fundamentales:

Nombre: El identificador único de la variable.

Tipo: El tipo de dato mapeado de AvengerScript a Java (ej. stark a int, banner a double).

Valor: El contenido almacenado.

Ámbito (Scope): Un sistema de rastreo (`currentScope`) que permite aislar las variables declaradas dentro de ciclos o funciones (ej. `global.for`, `global.while`) para evitar colisiones lógicas de memoria.

El análisis semántico es fundamental en instrucciones como la de lectura (`gamora`). Al invocar `gamora(edad)`, el `EvalVisitor` busca la variable en la tabla de símbolos, determina su tipo nativo y escribe dinámicamente el método correcto de la clase `Scanner` de Java (`scanner.nextInt()`, `scanner.nextDouble()`, etc.).

4. Traducción Dirigida por la Sintaxis

El recorrido del árbol permite concatenar el código traducido en un StringBuilder global (javaCode). Las principales traducciones se mapearon de la siguiente manera:

Construcción Avenger	Traducción Java	Justificación Técnica de la Transformación
jarvis (Asignación)	tipo id = valor;	Se infiere el tipo desde la gramática y se actualiza o registra en la tabla de símbolos.
loki (Ciclo While)	while (condicion)	Se extrae la condición recursivamente, se incrementa la indentación y se modifica el scope actual a nombreScope.while.
fury (Ciclo For)	for (init; cond; act)	Se mapea posicionalmente: expr(0) para la inicialización, la condition, y expr(1) para la actualización iterativa.
vision / wanda (If/Else)	if (cond) { } else { }	Se detecta la posición física del token WANDA dentro de los hijos del nodo sintáctico para dividir dinámicamente el bloque de sentencias en verdadero (antes de Wanda) y falso (después de Wanda).
Funciones	public static tipo nombre	Las producciones StmtFuncDecl y StmtFuncDeclVoid (con la palabra clave bob) se extraen como métodos independientes en la clase Java.

5. Importación de Librerías Externas (ASSEMBLE)

Para cumplir con el requisito de modularidad, se implementó la producción #StmtAssemble, la cual permite importar otros archivos (.avng o .txt) dentro del programa principal. La implementación en el EvalVisitor utiliza técnicas avanzadas de lectura y parseo recursivo:

Normalización de Rutas: Se utiliza `Paths.get(ruta).toRealPath()` para obtener la ruta absoluta del archivo externo.

Prevención de Ciclos Infinitos: Se mantiene un `HashSet<String>` llamado `archivosEnsamblados`. Antes de procesar un archivo, se verifica si ya existe en el set. Esto evita que si el archivo A importa al archivo B, y B importa a A, el compilador colapse en un ciclo infinito.

Visitor Hijo: Se instancia un nuevo `AvengerLexer` y `AvengerParser` en tiempo de ejecución. Luego, se crea un `EvalVisitor` secundario (hijo) que comparte la misma tabla de símbolos por referencia. De esta forma, las variables y métodos declarados en la librería externa se vuelven accesibles en el flujo del archivo principal, fusionando el código limpio bajo el comentario `// --- inicio: [ruta] ---`.

6. Conclusiones

El desarrollo de la fase II comprobó la robustez de la gramática diseñada en la fase I. Mediante la correcta eliminación de recursividad por la izquierda, ANTLR fue capaz de generar Árboles de Análisis Sintáctico libres de ambigüedad.

La implementación de un patrón Visitor, acoplado con una Tabla de Símbolos y manejo de scopes, permitió realizar un análisis semántico efectivo. Finalmente, las técnicas de recursión para importaciones de archivos y el mapeo estructural demostraron que el equipo asimiló con éxito la traducción dirigida por la sintaxis, logrando un transpilador temático plenamente funcional.